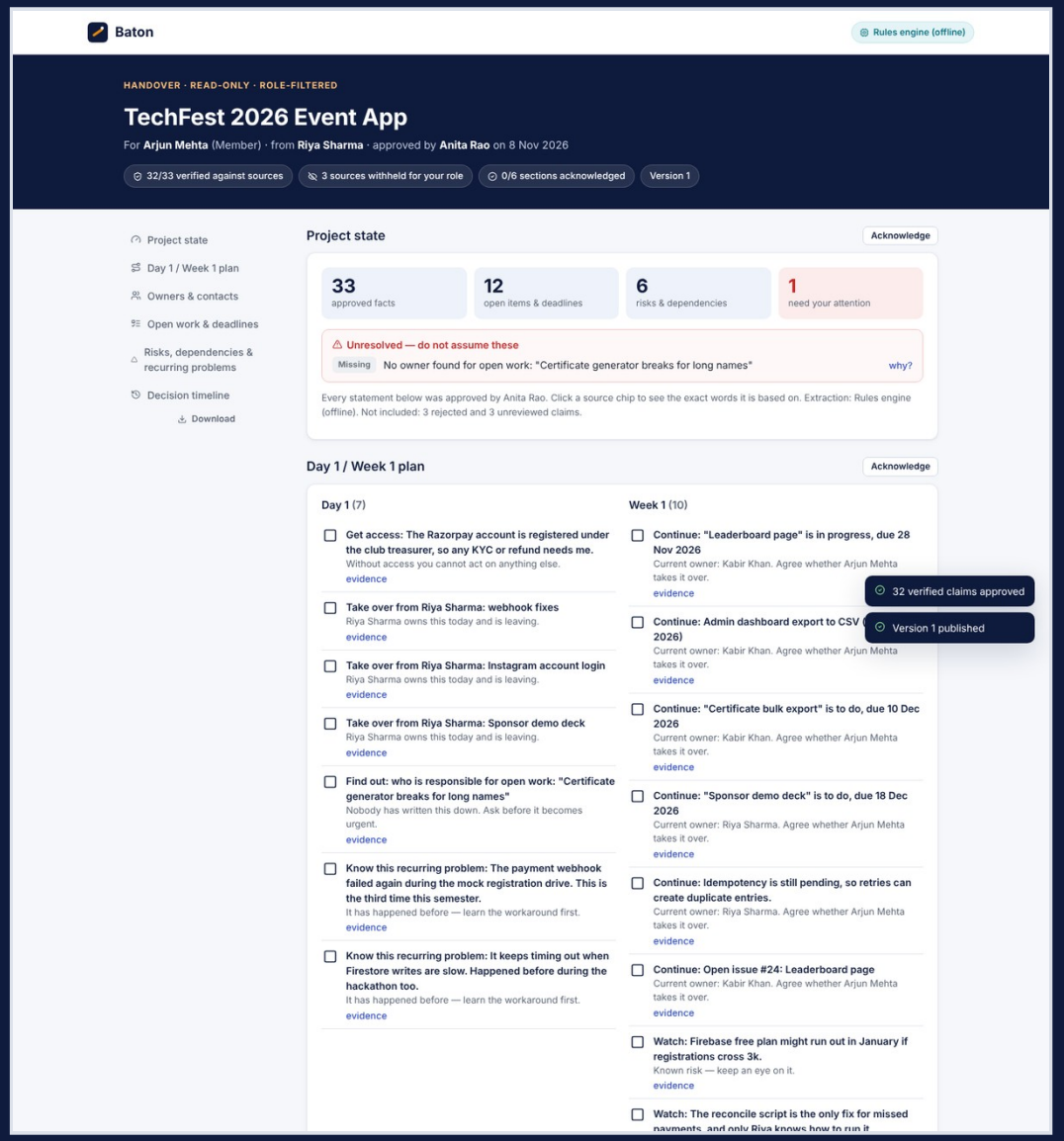
Baton

Evidence-checked, permission-safe project handovers — now a working product

TEAM	ATHRAV
LIVE PRODUCT	[https://your-project.vercel.app]
SOURCE CODE	[https://github.com/your-username/baton]
TRY IT	Open the link → Try the sample project (no sign-in)



Everything in this document describes software that is built and tested. All demo data is synthetic.

WHAT WE BUILT

A judge can go from raw project sources to an approved, evidence-linked handover in three minutes

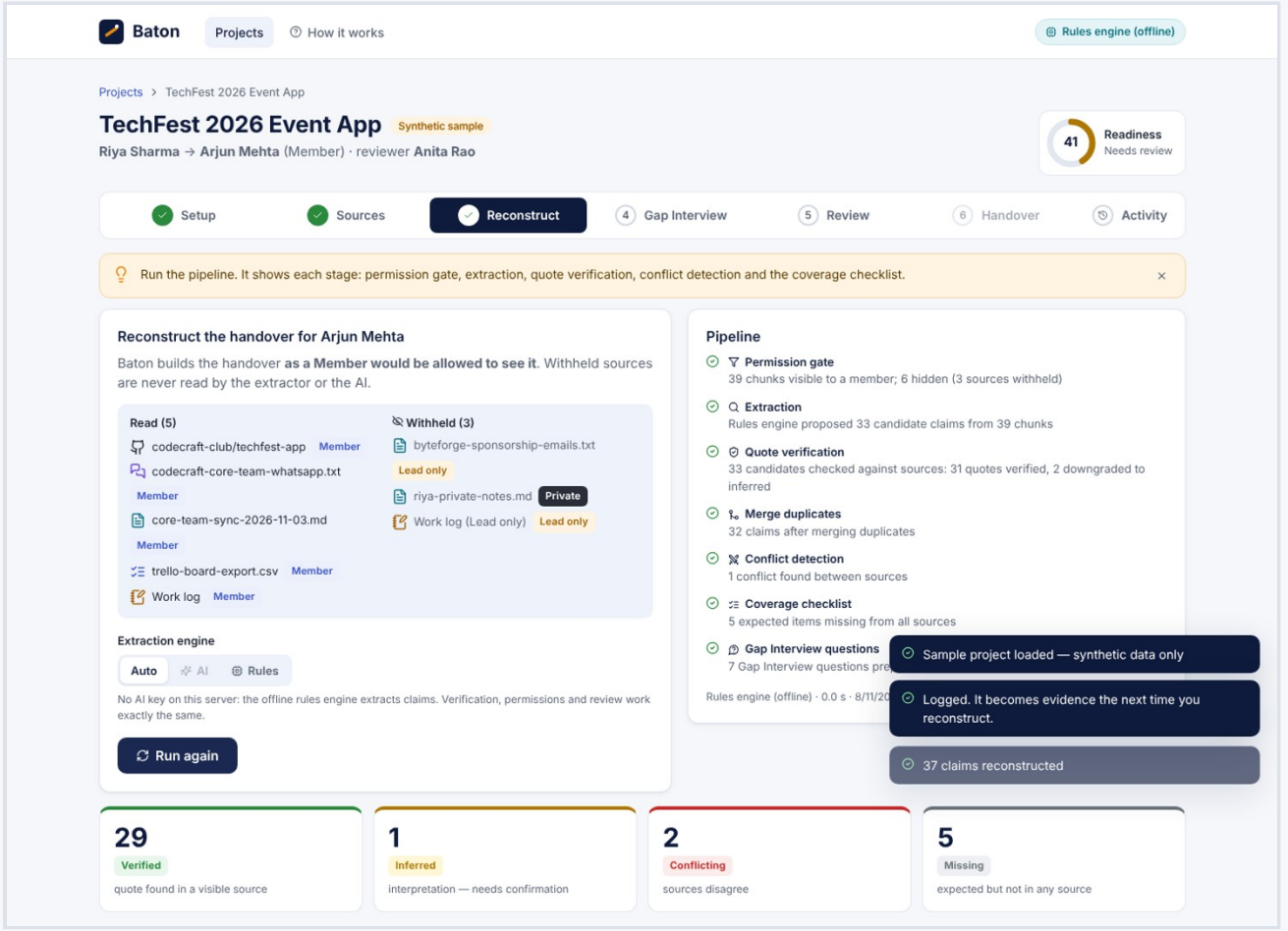
1. **Sources**
   Synthetic GitHub repo, WhatsApp export, notes, task board, sponsor emails (Lead only), private notes, work log — secrets redacted on arrival

2. **Reconstruct**
   Streamed 7-stage pipeline: permission gate → extraction → quote verification → merge → conflicts → coverage → questions

3. **Gap Interview**
   Leaver answers only what sources could not settle; answers become cited testimony

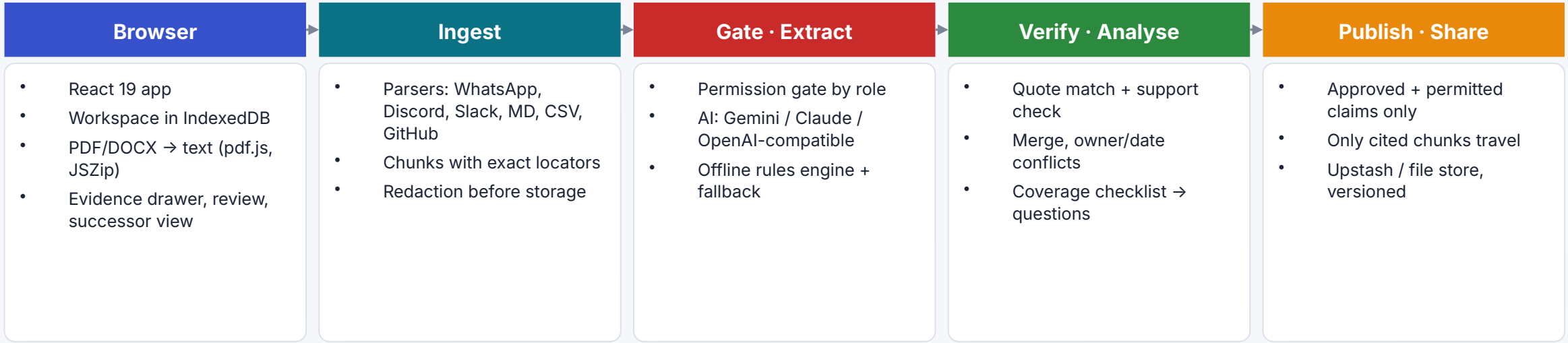
4. **Review**
   Exact highlighted evidence for every claim; approve, edit or reject

5. **Handover**
   Role-safe successor view, Day 1 / Week 1 plan, acknowledgement, flag as wrong



ARCHITECTURE AND DATA FLOW

Local-first browser app, one TypeScript API, a permission gate before the AI and a verifier after it



KEY TECHNICAL DECISIONS

Decision	Why
Status set by code, not the model	A citation is not proof; a string match against stored text is checkable
Gate before the AI	The model cannot leak what it never receives
Local-first workspace	Raw chats and notes stay in the user's browser; server stores only approved packages
Rules-engine fallback	Demo never breaks without a key, on quota limits or outages
Committed build output	Deploys with no install or build step — the judged link is reproducible

AI LAYER AND EVIDENCE VERIFICATION

The model only proposes claims; code decides how much to trust each one

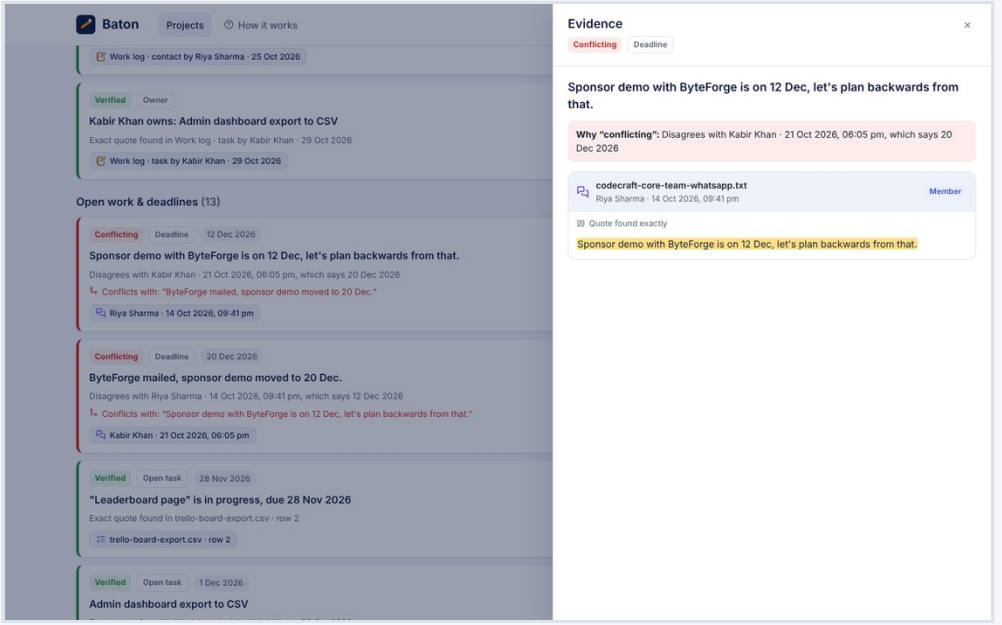
Step	Done by	Guardrail
Permitted chunks sent with ids	Code	Only chunks the successor's role may see; secrets already redacted
Extract typed claims (JSON)	AI model or rules engine	zod schema; one retry on invalid JSON; malformed claims dropped and reported
Quote check	Code	Exact → normalised → ellipsis → punctuation-insensitive; wrong chunk id relocated only among visible chunks
Support check	Code	Named people must appear in the evidence; ≥1/3 of statement words must appear in the chunk
Conflicts + coverage	Code	Owner/date disagreements shown side by side; checklist gaps become Missing
Failure handling	Code	Failed AI batches fall back to the rules engine with a visible warning

VERIFIED
quote found + supported

INFERRED
interpretation / check failed

CONFLICTING
sources disagree

MISSING
no source covers it



Evidence drawer: the exact quote is highlighted inside the stored source text, with the reason for the status.

Providers Google Gemini API · Anthropic Messages API · OpenAI-compatible Chat Completions (Groq, OpenRouter, local) — chosen by environment variable; keys never reach the browser. No key → offline rules engine.

PERMISSIONS, PRIVACY AND SECURITY

Least access by default — and proven absent on the successor's page by automated tests

Access tags per source
Member · Lead only · Private. A chunk is never more visible than its source; Private is never shared.

Permission gate before extraction
The extractor and AI receive only chunks the successor's role may see.

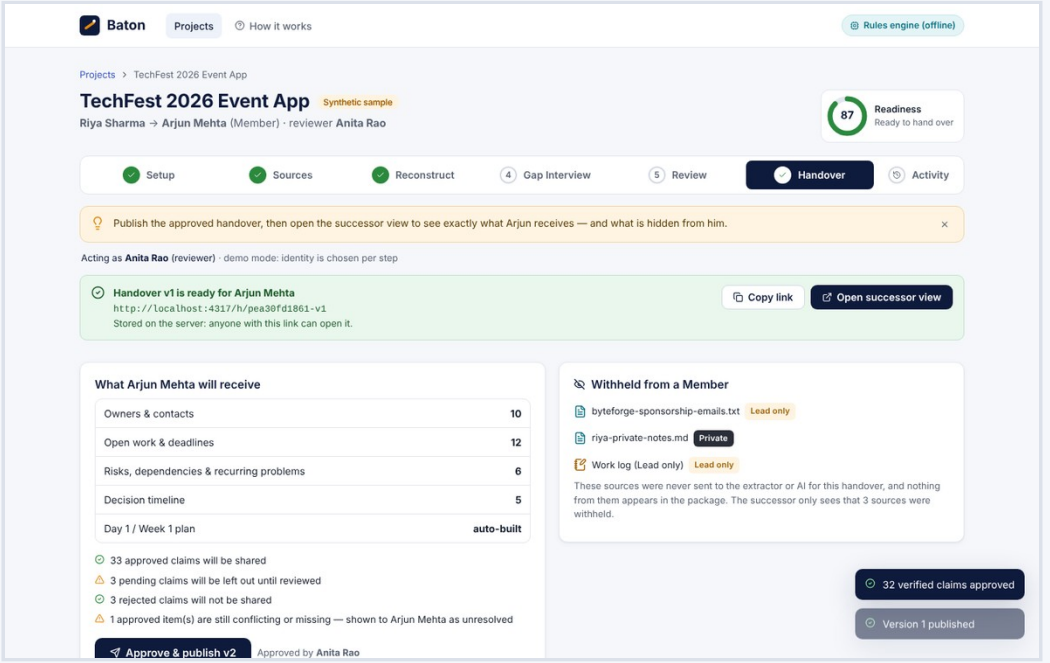
Publish-time check
An approved claim is dropped if any cited chunk is not visible to the role; only cited chunks travel.

Redaction on ingestion
Keys, tokens, KEY=value secrets, passwords, connection strings, Luhn-valid cards, emails, phones.

Local-first storage
Sources and drafts stay in the browser (IndexedDB); the server stores only published packages.

Hardened API
zod input validation, 8 MB body limit, per-IP rate limits, strict CSP, no third-party scripts, X-Frame-Options DENY.

Human control + audit
Nothing is shared before a reviewer approves; every action is logged with actor and time.



Leak tests (automated, passing)
Lead-only sponsor amounts absent for a Member · private key, password and private note absent · AI prompt for a Member contains no Lead-only/Private text · Lead sees Lead-only but never Private.

COMPLETED FEATURES

Every Round 1 Must-have works end to end — plus three Round 3 items and continuous capture

Round 1 plan item	Status in Round 2
Connectors: GitHub, documents, chat exports	✓ Built — plus Discord/Slack JSON, PDF, DOCX, task-board CSV, paste
Access tags per source	✓ Built — Member / Lead only / Private
Typed claim extraction	✓ Built — 9 types; AI + offline rules engine
Quote verification + 4 evidence states	✓ Built — status decided by code
Evidence viewer	✓ Built — highlighted quote, locator, source link
Review: approve / edit / reject	✓ Built — bulk approve, notes, readiness score
Role-safe successor view + Day 1 / Week 1 plan	✓ Built — acknowledgement, flag, Markdown export
Secret redaction	✓ Built — 8 categories
Gap Interview (planned Round 3)	✓ Built early — answers become cited testimony
Conflict detection + timeline (planned Round 3)	✓ Built early — owners and dates
Versions + acknowledgement (planned Round 3)	✓ Built early — version diff, per-section receipts
Task-board import (stretch)	✓ Built — CSV from Trello / Notion / Jira
New: Work log (continuous capture)	✓ Built — decisions with reasons, risks, tasks, contacts

CHANGES FROM THE ROUND 1 PLAN

Simpler stack, stronger guarantees — each change has a reason

Round 1 plan	Round 2 build	Why
Next.js + FastAPI + Supabase	React SPA + one TypeScript API (Node / Vercel)	One language, one deployment; same core code in server and tests
Postgres Row-Level Security	Permission gate in pipeline + publish-time check; local-first storage	No server-side workspace yet; RLS returns with accounts in Round 3
pgvector + embeddings	Batched extraction with exact locators	Student projects fit a few batches; exact quotes matter more than similarity
Claude API only	Gemini / Claude / OpenAI-compatible + offline rules engine	Free-tier option; never breaks without a key
Gap Interview, conflicts, versions in Round 3	Built in Round 2	Central to trust; Round 3 now adds new capabilities
—	Work log for continuous capture	Round 2 brief: capture knowledge while work happens
Real sign-in	Identity chosen per step (demo mode)	Judges must use the link without sign-in; accounts are a Round 3 commitment

TESTING EVIDENCE

59 automated checks pass on the final build — including permission-leak and failure cases

20/20
core unit + integration tests

5/5
AI provider contract tests

27/27
browser end-to-end checks (Playwright)

7/7
AI path over HTTP (mock model)

Failure case	Behaviour	Result
Model invents a quote	Downgraded to Inferred with reason	✓ Pass
Model cites a hidden / unknown chunk	No evidence attached; Inferred	✓ Pass
Prompt injection inside a source	Cannot change status	✓ Pass
Invalid JSON from model	Retry, then rules-engine fallback	✓ Pass
Model unreachable	Whole run falls back with visible warning	✓ Pass
Malformed request / unknown route	HTTP 400 / 404	✓ Pass
Sources or role changed after run	"Results are out of date" banner	✓ Pass

Structured feedback
[Add 3–5 real notes from students who tried the deployed link, and what you changed because of them. Remove this box if you have none.]

TECHNOLOGY, APIS, MODELS AND DISCLOSURES

Everything we used, and why

Layer	Used	Why
Frontend	React 19, TypeScript, hand-written CSS, react-icons (Lucide), Inter font (OFL)	Fast, accessible review UI
Backend	Node.js HTTP server / Vercel function, TypeScript, zod	One handler for local and serverless; validated inputs
AI models / APIs	Google Gemini API, Anthropic Messages API, OpenAI-compatible API — configurable	JSON output, long context, free tier available
Integrations	GitHub REST API (read-only), Upstash Redis REST	Repository import; durable share links
Parsing	pdf.js, JSZip (in browser); custom WhatsApp/Discord/Slack/CSV/Markdown parsers	Exact locators for citations
Build & test	esbuild, TypeScript, tsx + node:test, Playwright	Fast builds; real-browser tests
Hosting	Vercel (or Render / any Node 18+)	Free, no build step needed

Disclosures No code existed before the hackathon. Datasets: synthetic sample data written by the team — no real people, club or company data. Templates: none. AI assistance: Claude (Anthropic) was used as a coding and writing assistant; the team reviewed and owns all product decisions. Open-source libraries are used under their licences.

THREE MAJOR IMPROVEMENTS FOR ROUND 3

New capabilities, not cosmetics — each removes a real limitation of today's build

1 Live connectors with continuous capture

Slack/Discord bot (/baton decision ...) and GitHub webhook log decisions, risks and ownership changes as they happen, with scoped read-only OAuth instead of manual exports.

Removes: manual exports; handover only at exit

2 Ask Baton — grounded successor Q&A

Plain-language questions answered only from approved claims and their cited text, always with citations, role-filtered, and "not in the handover" when no evidence exists.

Removes: successor still has to scan everything

3 Real accounts and organisation access control

Sign-in, invitations and per-person roles; server-side workspaces with Postgres Row-Level Security; signed share links with expiry and revocation.

Removes: demo-mode identity; browser-only workspaces

KNOWN LIMITATIONS TODAY (HONEST)

Identity chosen per step (no accounts) · rules engine is English-only and pattern-based · conflicts cover owners and dates · files up to 4 MB · no OCR for scanned PDFs · share links on other devices need Upstash configured.

TEAM ATHRAV

Who built what

Name	Role	Built in Round 2	Round 3 ownership
Bharat [surname] (lead)	[role]	[e.g. AI layer, verifier]	[e.g. Ask Baton]
[Member 2]	[role]	[e.g. ingestion, GitHub]	[e.g. live connectors]
[Member 3]	[role]	[e.g. review + successor UI]	[e.g. Q&A UI]
[Member 4]	[role]	[e.g. permissions, tests]	[e.g. accounts + RLS]

Links Live product: [deployed URL] Repository: [GitHub URL] Docs in repo: README · docs/ARCHITECTURE.md · docs/TESTING.md · docs/DEPLOY.md
Every link opens without sign-in — tested in an incognito window.